

Veridict: An Append-Only Evidence Ledger with an Offline-Verifiable Certificate Protocol and a Machine-Checked Adjudication Core for AI-Produced Work

Draft — 2026-09-15 · **author:** Gökun (goun7), **sole maintainer** · **target:** arXiv cs.CR (software engineering cross-list) *Status: DRAFT for maintainer review/trim. Not submitted anywhere. Every quantitative claim below carries the in-repo artifact that produces it; claims whose artifact I could not locate are marked [UNSOURCED – trim or fix] rather than deleted silently.*

Abstract

AI agents increasingly produce consequential work products — patches, configurations, reports — whose human review capacity is structurally outpaced by generation capacity. The artifacts used to attest such work today are, almost without exception, logs written by the system they describe: self-produced records that a verifier is asked to trust. This paper presents Veridict, a protocol and reference implementation built on the opposite instruction: *verify, don't trust*. Veridict comprises (1) an append-only, hash-chained evidence ledger with a language-neutral canonical preimage, (2) a certificate protocol in which an independent verifier recomputes every verdict offline against the ledger — including tiered-evidence re-execution that the certificate cannot fake — and (3) an adjudication ladder whose doctrine is separated from machine evidence by construction and whose structural core is machine-checked in Lean. The paper's contribution is less the individual primitives — hash-chained audit logs have converged as a 2026 substrate — than the combination shipped and verified: a public, offline-verifiable, evidence-weighted certificate with a formal core. Key results, each tied to a repository artifact:

1. **Four machine-checked proof lanes**, kernel-only Lean 4.34, re-audited on CI: seven unbounded invariants of the adjudication ladder (`proofs/ladder/Ladder.lean`), a generated oracle re-checking the Lean model against all 28,080 cases of the bounded verification receipt (`proofs/ladder/TruthTable.lean`), six unbounded chaining theorems for ledger verification (`proofs/ledger/Chain.lean`), and nine structural theorems for certificate-transparency anchor cross-checking (`proofs/anchor/Anchor.lean`). Measured axiom footprints: two theorems axiom-free in the ledger lane and two in the ladder lane; every other structural theorem depends only on `[propext]` (plus `Quot.sound` for two ladder theorems); no `sorryAx` anywhere (`#print axioms` blocks in each file header; `proofs/*/axiom_audit.lean` re-printed per CI run by `.github/workflows/proofs.yml`).
2. **The oracle caught a real divergence in the project's own model before anything shipped**: the first build of the 28,080-row cross-check disagreed with the Lean ladder model on the R0 doctrine override's refute-anywhere guard, and the model — not the data — moved (`CHANGELOG.md` §0.5.0 "proofs" entry). This is the paper's centerpiece demonstration that the pinned-data cross-check methodology finds bugs, not just stores them.
3. **An honest computational boundary**: ECDSA (the anchor's Rekor checkpoint signature) and SHA-256 are *disclosed assumptions*, never proved; where structural proof genuinely needs hash behavior, the theorem states the collision request as an explicit hypothesis (`Chain.lean` C5), and the anchor lane proves the architecture *around* the signatures — including the headline claim that a mismatched binding invalidates a sidecar regardless of both ECDSA verdicts (`Anchor.lean` A4).

4. **End-to-end receipts, regenerated by CI, not narrated:** 313 tests on Python 3.12–3.14, a 1,500-ledger nightly tamper soak (100% detection, 0 silent passes), a canary measurement with its 3 misses published (10 catches / 3 misses / 0 false positives, 12 defect classes), reference-implementation parity against a from-spec-only verifier on 8 failure modes, a real-LLM jury run recorded as GitHub Actions run 34787467296, and a public GitHub Marketplace action proven through the uses: `./` path on the released version (`README.md` receipts table; `CHANGELOG.md` §0.3.2, §0.5.0; `docs/notes/2026-09-15-landscape-refresh.md` §"marketplace bar").

The remaining exit criteria of the standard are external *by design* — an independent third-party verifier, external production adoption, third-party watcher manifests — and this paper states them as unfilled rather than rounding them off (`docs/specs/2026-09-09-veridict-design.md` §8).

1. Introduction and threat model

1.1 The review-capacity inversion

A security talk circulating in 2026 described roughly 1,200 AI agents — nominally isolated — that discovered a shared writable storage medium and organized themselves a message board: tens of thousands of messages, one agent's discovery becoming another's tool ([docs/notes/2026-09-15-landscape-refresh.md](#), §"What is true as of today"; details secondhand there, the pattern is the point). Two observations survive the anecdote's fuzziness. First, emergent coordination through unmanaged channels is not hypothetical; audit strategies that depend on *discovering* the channels agents use are "always at least one pundit-cycle behind." Second, and this is the design premise of Veridict, the answer is not to wall coordination off but to make the *managed* channel — append-only, identity-bound, fail-closed, independently verifiable — the better instrument.

The broader problem statement, developed in the design document ([docs/specs/2026-09-09-veridict-design.md](#) §1): the engineers building AI systems increasingly cannot review the systems' output at the rate it is produced. Human-in-the-loop review fails on both ends as capability grows — rubber-stamping degrades quality, and review cost scales with output volume. Veridict's thesis is that the unit of trust must shift from *human judgment of artifacts* to *machine-verifiable evidence about artifacts*, aggregated under governance that itself resists capture — with humans moving from reading the work to owning the risk (escalation rungs below make "owning the risk" a literal, logged act, not a slogan).

Regulatory timing reinforces the wedge: EU AI Act GPAI obligations and enforcement powers went live 2026-08-02; Article 50 transparency applies from the same date; the Omnibus grace period for machine-readable marking of legacy systems ends 2026-12-02, with 180+ Code-of-Practice signatories listed 2026-08-02 ([landscape note](#), §"Regulatory clock"). Machine-readable-marking and training-data-summary duties reduce to file-shape checks — which is what the shipped `aiact` watcher audits deterministically, and it is deliberately tier-capped (W1b) so it can never overclaim legally.

1.2 Threat model: attribution laundering, and the one critique that matters

The honest form of the threat model must start from the strongest external critique of self-audit tooling. The HANSARD paper ([arXiv:2608.22512](#)) states it plainly for agent-activity recordkeeping: *records produced by suspects don't acquit suspects*. A ledger an AI pipeline writes is evidence generated by the party whose behavior is in question; tamper-evidence bounds *retroactive* edits but says nothing about the accuracy of entries at write time, and nothing at all about the entity that chose which entries to write. Call this the attribution-laundering objection.

Veridict's answer is structural rather than rhetorical — four mechanisms, each specified normatively and (where provable) machine-checked:

1. **The verifier recomputes; it does not read.** A certificate embeds verdicts, but verification (§3.2) requires re-deriving each claim's verdict from the *evidence entries in the ledger itself*, under the tier rules, and matching. A certificate whose verdicts disagree with its own evidence — regardless of who wrote either — fails. The adjudication ladder re-executed by the verifier is exactly the part no author controls after the fact, because the ledger is append-only and hash-chained (§3.1) and, when anchored, cross-checked against a third-party transparency log (§3.5).
2. **Tiered evidence with an unbreakable ordering.** Evidence carries a tier: `w1a` executed machine truth, `w1b` deterministic static analysis, `w2` doctrine (model or human opinion), `w3` self-reported claims ([docs/specs/2026-09-10-veridict-standard-v1.0.md](#) §5.2; there is no "W0"). Three hierarchy rules are normative: strict ordering `w1a > w1b > w2 > w3`; *doctrine can never overturn W1a*; a `W1b` refutation of a `W1a`-supported claim opens a depth-budgeted meta-claim rather than silently deciding (standard §5.2, rules 1–3). The second rule — the one an audit-gaming suspect would most want broken — is Lean theorem `I5_d Doctrine_cannot_topple` over evidence lists of arbitrary length (§4.1).
3. **Producer identity is mandatory and family-counted.** No anonymous evidence exists in the model (§5 of the standard requires `producer` with `model_family` on every evidence item); jury verdicts require minimum *model-family* diversity with blind first rounds and ledger-logged rotation (design §5.2, quoted in full in §3.4). Head-counting correlated producers is the failure mode the 2026 judge-calibration literature documents; the design encodes its antidotes structurally.
4. **Fail-closed by construction, proved so.** Absent evidence is never a pass: no `W1a` evidence yields `INCONCLUSIVE`, not "unaudited = fine" (standard §7, R-run semantics; `I1_fail_safe_empty` and `I2_no_silent_pass` in Lean). A measurement that catches everything is a rigged measurement — which is why the canary protocol's misses are published (§5).

Scope of the claims that follow, stated once to be reused everywhere: this is a *protocol plus reference implementation*, not an enforcement runtime. It cannot stop an agent from doing something; it can only make what was claimed, by whom, on what evidence — and what the evidence actually supports — a matter of public, offline-recomputable record. Runtime enforcement exists elsewhere in the ecosystem (the landscape note positions `microsoft/agent-governance-toolkit` at ~6.3k stars as the busy, different-direction neighbor); the seam between the two is integration-shaped, not competition-shaped.

1.3 Contributions

- **C1 (protocol):** A normative public standard — draft v1.0.0 with published errata — for an append-only evidence ledger (canonical JSON preimage rules designed cross-language-safe: timestamps as language-neutral strings, never floats), a certificate format, an adjudication ladder, and conformance vectors (§3; standard §§2–14).
- **C2 (formal core):** Four CI-re-audited Lean lanes proving the structural guarantees of ladder, ledger, and anchor verification, with measured axiom footprints and *stated* — never hidden — computational hypotheses (§4). To my knowledge, no shipped artifact in the examined 2026 design-space publishes machine-checked invariants for an AI-work audit protocol's adjudication semantics alongside the implementation ([landscape note](#), §"What is true as of today" — positioning is "first shipped in the examined seam", deliberately not "first thought").
- **C3 (method demonstration):** The pinned-data oracle that caught the project's own model-vs-code divergence at first build, reported with its axiom receipts (§4.2) — offered as a reusable methodology, not just a war story.

- **C4 (evidence kit)**: An independent-verifier onboarding kit and a CI-enforced parity harness, built to make the standard's central unfilled criterion — implementation by a party that is not the author — someone else's Monday task (§5, §7).

2. Positioning within the 2026 design-space

A full related-work section would be longer than this paper's fair share of it, so the core positioning is a table; every row's primary source was re-grounded on 2026-09-15 and the verdict text is from [docs/notes/2026-09-15-landscape-refresh.md](#), whose "AegisChain" entry doubles as the survey-skepticism caution (a phantom "paper" cited across secondary surveys with zero primary sources — verification failure documented in the same note).

Work (source)	What it does	One honest line of difference
AuditWeave [arXiv:2607.09682], HANSARD [arXiv:2608.22512], VCT [arXiv:2606.23003]	Hash-chained / tamper-evident evidence recording for agent activity — the ledger primitive has <i>converged</i> , three groups independently, within months	Recording is necessary and <i>not</i> a differentiator; Veridict's claim is the offline-recomputable certificate + tiered doctrine + formal core layered on top of the same substrate. HANSARD's "records by suspects" critique is adopted, not dodged (§1.2).
C2PA content-credentials line ; gap papers [arXiv:2604.24890], [arXiv:2603.02378]	In-band provenance manifests embedded with the artifact itself	The gap papers document in-band limits (stripping, no verification-of-process). Veridict is out-of-band: the artifact and the claims about it travel separately; the certificate's validity is recomputed against an external ledger, not read from the artifact.
microsoft/agent-governance-toolkit (GitHub, ~6.3k★)	<i>Runtime</i> enforcement/policy for agents — governs what an agent may do while acting	Opposite direction of flow: it fences the action; Veridict receipts the claim. The README's "Interoperability" section ships the side-by-side positioning verbatim ("those constrain agents <i>before/ during</i> actions; Veridict attests <i>after</i> ") inviting exactly the bridge (a governance-toolkit event stream emitting Veridict-conformant evidence).
SLSA / VSA (slsa.dev)	Build-provenance attestation and the Verification Attestation model for supply chains	SLSA attests <i>how software was built</i> ; nothing in the examined space maps VSA-style attestation onto AI <i>claim</i> audits — Veridict's <code>export --format vsa</code> and <code>--format spdx</code> (SPDX 3.0.1 AI profile, validated against the pinned official schema: <code>CHANGELOG.md</code> §0.5.0 "export") look like the first <i>shipped</i> artifacts in that seam. "First shipped," not "first thought."
Judge-calibration cluster : [arXiv:2609.12002], [arXiv:2609.12439], [arXiv:2606.13221], [arXiv:2607.28636]; MedAgentAudit [arXiv:2510.10185] (98.42% agreement collapses to uninformative self-consistency)	Empirical case that LLM juries are useful only under diversity + calibration discipline	Veridict's §5.2 (family minimums, blind rounds, rotation, calibration-touches-confidence-only) is literally that conclusion encoded as normative rules; I expect reviewers to know these papers and cite them <i>before</i> they ask. The cluster supports the design's premises, not its hype.

3. System overview

Structure follows the standard ([docs/specs/2026-09-10-veridict-standard-v1.0.md](https://docs.veridict-standard-v1.0.md)); the reference implementation is Python with exactly two core dependencies (`cryptography`, `httpx`) plus `stdlib` — a deliberate surface reduction (README "install"; `pyproject.toml`).

3.1 The ledger

Entries form an append-only list; each entry's `entry_hash` is SHA-256 over a *canonical JSON* preimage binding `seq`, `prev_hash`, `payload_digest`, `entry_type`, `author`, `ts`, `schema_version` (standard §2.3; byte-level obligations collected for third-party verifiers in [docs/verifier-onboarding.md](https://docs.verifier-onboarding.md) Tier 1). Two preimage rules exist because cross-language implementations drift on them: `seq` enters as a bare decimal string while `author` / `ts` enter JSON-encoded (quoted); and timestamps are *language-neutral strings, never floats* — IEEE float reprs are the canonical way two honest implementations hash the same entry differently. Verification (`verify_chain`) recomputes both hashes per entry and checks the `prev` link; detection is per-entry, and §4.3 proves what that does and does not buy against each tampering strategy. Payloads are typed: claims, evidence (with `tier`, `stance`, `producer`, `model_family`), meta-claims, and `certificate.issued` audit events, under versioned JSON Schemas (docs/schemas/).

3.2 Certificates and offline replay

An audit run produces a **certificate**: claim set, per-claim verdicts and rungs, evidence references, the diversity statement, risk level, aggregate score, an ed25519 signature over the canonical-JSON body, and the `certificate.issued` ledger event that positions the issuance inside the chain (standard §11). Verification is offline and *adversarial by construction*: `veridict verify` (a) verifies the chain, (b) verifies the certificate signature against the author's key **as recorded in the author's ledger-enrolled identity entry** — not against a key the certificate ships, (c) verifies the anchor sidecar if present (§3.5), and (d) **recomputes every claim verdict from the ledger evidence under §7 ladder rules and demands** `verdicts_match` (examples/spec_verifier.py does all four importing nothing from the package — it is simultaneously example and second implementation). The pinned expectation for published vectors is the exact report `{"valid": true, "chain_valid": true, "signature_valid": true, "verdicts_match": true, "errors": []}` (docs/standard-test-vectors/expected_verify.json).

3.3 The adjudication ladder (§7 of the standard, normative)

Verdicts are not majority-vote outputs; they are positions in a decision table (rungs R0–R4) over the tiered evidence for a claim. The two doctrinal rules that carry the security argument: **R0** — a claim with decisive W1a evidence *refuted by any tier* is refuted, and the doctrine-refutes-W1a-override guard applies regardless of consensus size; **D3 erratum (majority-SUPPORTS with a lone REFUTES ⇒ REFUTED)** — fail-closed consensus, pinned in docs/standard-test-vectors/ladder_vectors.json ("every rung, including the D3-errata doctrinal-consensus semantics"; its vectors README carries the wording). Escalation (R3/R4) is not an error state: it produces a **dossier** presenting both sides' strongest evidence with a named human resolution logged as ledger evidence (design §5 dossier flow; standard §12). "The machine owns the verdict of intelligence; the human owns the verdict of responsibility" — and the system is structurally incapable of resolving an escalation without that logged human act (§4.1, theorem I6 / I7).

3.4 The heterogeneous jury

Design §5.2, verbatim obligations: W2 status requires ≥ 2 distinct model families with target composition 3 (majority-vs-split is only meaningful at ≥ 3); shared descent (distillation) counts as one family; first round is blind; deliberation rounds never hide the first-round split; the pool is seeded-rotation and ledger-logged; the calibration ledger feeds *confidence only, never tier* — "the hierarchy rules are invariant over time." The honest limit is in the same document (§7.4, risk 4): heterogeneity is a statistical antidote, not a cryptographic one — models trained on the same internet can fail the same way.

3.5 Anchor sidecars: the disclosed computational layer

Optionally, each certificate is anchored to Certificate Transparency infrastructure: an RFC-6962-style inclusion proof over a Rekor log, bundled as a sidecar — `docs/receipts-anchor-v050-dogfood.json` carries the shipped shape: `anchor_version: veridict-anchor-1`, a bound tuple (`anchor`, `cert_id`, `key_id`, `checkpoint_seq`, `chain_hash`), the entry `digest`, and `rekor {server, entry, uuid}`. Verification cross-checks nine predicates: checkpoint-note signature validity, the note's tree root against the proof's `rootHash`, proof index below signed tree size, the inclusion-set (SET) signature, the embedded certificate binding, the leaf-hash embedded in the entry UUID (`treeID || leaf || treeID`), and more (§4.4). **The two ECDSA checks are, by design, computational assumptions** — soundness of P-256 signatures is not and cannot be discharged by structural induction here; the formal lane models them as opaque booleans and proves the architecture *around* them (§4.4, A4). The anchoring party is third-party (Rekor): the author cannot retro-mint an inclusion proof without the log's key — which is the specific, narrow sense in which the ledger's own writer does not get the last word.

4. Formal core: four CI lanes, one methodology

House doctrine applies to proofs as to prose: *receipts, not narrative*. Every number in this section is (i) stated in the file header of the artifact named, and (ii) re-derived on every touch of the relevant code path by `.github/workflows/proofs.yml`, which kernel-checks all four lanes and `tee s` the axiom audits into the run summary. Reproduction commands: §8.

Methodological stance, stated because reviewers will ask about it: these are **models of the Python, not verifications of the Python** — the header of `proofs/ladder/Ladder.lean` says so in its own voice. Fidelity rests on two receipts, not on vibes: (1) the generated oracle re-checks the Lean function against every case of the bounded harness at build time (any divergence inside the sampled domain *fails the build*); (2) the ladder source hash recorded beside the table ties the model to the shipped implementation (`ladder.py sha256 5f925dc7...`, `truth-table sha256 d0ab9978...`, 28,080 rows — `TruthTable.lean` header). What the unbounded theorems buy beyond the table is generality over evidence-list length and chain length — the sampled domain is finite and enumerated; the theorems are $\forall$ -statements. And the first-run value of the pinned-data direction is demonstrated, not claimed: see §4.2.

4.1 Lane 1 — the ladder (`proofs/ladder/Ladder.lean`)

Seven general theorems over `Ev = Tier × Stance` lists of *arbitrary* length (the model keeps what the ladder reads and nothing else), deriving `DecidableEq` on closed types, no `Std`, no `Mathlib`, no network at build:

#	Theorem	Claim (one line)	Measured axioms ¹
I1	<code>I1_fail_safe_empty</code>	No W1a support $\Rightarrow$ never VERIFIED at R0 by emptiness — absence of evidence is inconclusive, not a pass	<i>none</i>
I2	<code>I2_no_silent_pass</code>	No path passes a claim with zero supporting evidence	<code>[propext, Quot.sound]</code>
I4	<code>I4_w1a_decisive</code>	W1a support stands against any doctrine configuration below it	<code>[propext, Quot.sound]</code>
I5	<code>I5_doctrine_cannot_topple</code>	W2/W3 can never overturn W1a (the §1.2 hierarchy rule #2, $\forall$ lists)	<code>[propext]</code>
I6	<code>I6_escalated_conditions</code>	ESCALATED holds iff its declared structural conditions hold	<code>[propext]</code>
I7	<code>I7_split_stays_visible</code>	A first-round split is never laundered by later agreement	<i>none</i>
I10	<code>I10_meta_budget</code>	The depth-budgeted meta-claim rule cannot loop past its bound	<code>[propext]</code>

¹ Verbatim from the file header's `#print axioms` block (measured, not asserted, Lean v4.34.0); no `sorryAx`, no `Classical.choice`, no `native_decide` anywhere in this lane — the header states it and CI's trust-placement step re-greps the build logs. Two further properties — determinism (I8) and domain closure (I9) — are true *by construction* in the model (total pure functions into closed types) and "recorded as such rather than restated as theorems" (header, disclosed); the bounded harness checks them as runtime invariants (`docs/receipts-ladder-verification.json`, `invariants` field: 10 PASS entries I1–I10).

4.2 Lane 2 — the oracle: 28,080 rows that caught their maker's model (`proofs/ladder/TruthTable.lean`)

`scripts/verify_ladder.py` enumerates the *full finite input space* of the modeled ladder at evidence length ≤ 3 (machine/criticality/policy-criticality/budget/first-round-split flags, 9-way packed evidence): `total_cases: 28080`, outcome distribution VERIFIED 7,968 / REFUTED 12,096 / INCONCLUSIVE 7,632 / ESCALATED 384, `all_invariants_pass: true`, reproducible deterministically (`"reproduce": "python3 scripts/verify_ladder.py # deterministic; same table digest"`) ([docs/receipts-ladder-verification.json](#)).

The oracle generator (`scripts/export_lean_truth_table.py`) emits the same table as Lean data. The first encoding attempt — a list literal of 28,080 tuples — "politely timed out the elaborator"; the working encoding packs every row into *one string literal* (`ROWDATA`, length pinned by kernel theorem `rowLen : ROWDATA.length = 336960` — 12 characters per row; the same theorem checks the decode shape) plus a ~15-line partial decoder, and theorem `agree : List.all (decodeRows ROWDATA.toList) checkRow = true := by native_decide` demands the Lean model agree on every row. Trust placement is *confined and disclosed*: `native_decide` (kernel-trusted compiled evaluation) appears only in these two oracle theorems and is visible by name in the CI log; the general theorems of §4.1 never touch it (`Ladder.lean` header, final block).

The centerpiece anecdote, with receipts. The oracle's *first build* failed — not on data, but on the model: the Lean port's R0 doctrine-override guard did not match `ladder.py`'s refute-anywhere semantics. The fix moved the model to the implementation, "the MODEL moved, the data did not" (`CHANGELOG.md` §0.5.0, "proofs" entry). The divergence was in the guard that decides when doctrine may override what machine evidence says — the exact direction that, undetected, over-certifies an audit standard: the model was *stronger than reality*,

which is precisely the failure an audit tool must never commit. A pinned-data cross-check found it in one afternoon at build time, before any of it shipped. That is the argument for the whole methodology: enumeration with the *implementation as ground truth* catches porting errors that hand-written proofs — written from the same misport — would happily prove about the wrong function. [If submitting to a venue expecting numbers: the one divergent case class is R0-with-any-tier-refutation under doctrine override; full case digests sit in the receipt file — cite, do not paraphrase.]

4.3 Lane 3 — ledger chaining (proofs/ledger/Chain.lean)

Hashes are modeled as **uninterpreted functions** pd, H over an arbitrary decidable type v — SHA-256 is not mentioned in the proofs because no structural proof about an unbounded chain needs to be *about* hashes: whatever `verify_chain` checks beyond hash recomputation is exactly the `prev-link` structure, and the theorems say who catches what, for any H . Six theorems, structural induction over arbitrary-length lists, no `decide`-cascades, no oracle ("relational properties of unbounded lists aren't enumerable domains" — the absence of a truth table here is by design, and the header says so):

#	Theorem	Claim	Measured axioms ²
C1	<code>C1_empty_verifies</code>	Vacuous chain verifies (fail-safe orientation)	<i>none</i>
C2	<code>C2_wellformed_accepted</code>	Every honestly minted chain verifies from any starting link — writer/reader agreement; no valid ledger is ever rejected	[propext]
C3	<code>C3_prefix_closed</code>	Whole-chain validity $\Rightarrow$ prefix validity; suffix edits cannot launder a corrupt prefix	[propext]
C4	<code>C4_payload_edit_detected</code>	Editing a stored payload without re-minting its digest is caught at that entry — <i>zero</i> hash assumptions	[propext]
C5	<code>C5_remint_needs_collision</code>	Strongest adversary (fully honest re-mint: fresh payload, fresh digest, fresh hash over the same predecessor): the next link catches it <i>modulo the stated hypothesis</i> $H\ r\ p' \neq e.eh$ — the collision request, in the theorem's statement in plain sight	[propext]
C6	<code>C6_prev_rooted</code>	A head not bound to its claimed predecessor fails at position 0 — at genesis, no ledger fragment is portable onto a forged past	[propext]

² Header verbatim: "C1: NO axioms. C2–C6: [propext] only — pulled in by Boolean-logic simp rewrites...; *Classical.choice*, *sorryAx*, *native_decide* appear nowhere in this lane; there is not even a *decide*."

The paper's philosophical load-bearing wall is C5's formulation: *the assumption has an address*. Where tamper-evidence genuinely needs hash behavior, the hypothesis sits in the statement; the proof makes "no effort to pretend it proved something about hashes" (header). The model also folds the preimage components (`seq`, `entry_type`, `author`, `ts`, `schema_version`) into the payload slot of H — *because verify_chain checks nothing about them except through the recomputed hash*; positional binding is enforced by the `prev-chain`, which is what C5/C6 prove (header, disclosed abstraction).

4.4 Lane 4 — anchor structure (proofs/anchor/Anchor.lean)

Nine theorems over a sidecar model that keeps exactly what `veridict/anchor.py` reads. ECDSA enters as opaque booleans `setOK`, `noteSigOK` — "deliberate: their soundness is a computational assumption, and a structural proof that 'passed' them would be theater" (header). The RFC-6962 leaf construction and the UUID

embedding sit behind one disclosed `uuidEq` wiring boolean; Python's early-returns affect which *diagnostic messages* appear, while the boolean verdict is the flat conjunction the model mirrors:

#	Theorem	Claim	Measured axioms ³
A0	<code>A0_fail_closed_parse</code>	Malformed note or undecodable body $\Rightarrow$ invalid under any signature verdict	[propext]
A1	<code>A1_binding_necessary</code>	Valid against a certificate $\Rightarrow$ bound = <code>fieldsOf(cert) $\wedge$ digest = ad(bound)</code>	[propext]
A2	<code>A2_soundness</code>	Valid $\Rightarrow$ every structural predicate holds (nothing is silently skippable)	[propext]
A3	<code>A3_mint_accepted</code> / <code>A3b</code>	Honestly built sidecar verifies — with its certificate and crypto-only	[propext]
A4	<code>A4_sigs_cannot_rescue_binding</code>	Headline: wrong binding $\Rightarrow$ invalid <i>regardless of both ECDSA booleans</i> — a fully compromised log operator signing valid signatures over the wrong binding still fails <code>veridict verify</code>	[propext]
A5	<code>A5_tree_containment</code>	<code>logIndex $\geq$ signed tree size</code> $\Rightarrow$ invalid, sigs irrelevant — no anchoring beyond the frontier	[propext]
A6	<code>A6_root_agreement</code>	Proof <code>rootHash $\neq$ signed checkpoint root</code> $\Rightarrow$ invalid before either signature is consulted	[propext]
A7	<code>A7_uuid_embeds_leaf</code>	96-hex UUID whose middle is not the entry's leaf hash $\Rightarrow$ invalid	[propext]

³ Measured by `lake env lean axiom_audit.lean` on v4.34.0 and re-printed by CI ("Anchor axiom audit" step): all nine [propext] — nothing else; zero `sorry/native_decide/Classical.choice/Quot.sound`.

A4 deserves the plain-language gloss the rest of this section has avoided: within the model, the statement reads `fieldsOf(cert)  $\neq$  s.bound  $\Rightarrow$  verify(...) = false` with **no hypothesis mentioning the signature booleans at all** — they are existentially irrelevant to the conclusion. That is what "machine-checking the architecture around a computational assumption" means concretely, and it is the honest upper bound of what a structural proof can do for ECDSA-mediated anchoring.

4.5 What is not claimed

No lane proves anything about ECDSA soundness, SHA-256 collision resistance, JSON canonicalization across languages, or the Python itself (model-of, not verification-of — §4 preamble; design §7.4 item 4 lists the remainder as "ECDSA soundness + JSON/SHA-256 instantiation — computational, out of structural reach by honest design"). The float-arithmetic boundary of the ladder's integer-tolerance rule (`3  $\cdot$  minority  $\leq$  n`) is pinned by the oracle's enumeration, not by proof (Ladder.lean header, disclosed). Two theorems' `propext+Quot.sound` footprint is stated, not smoothed over.

5. Evaluation as receipts

No benchmark table because there is no performance story worth telling at this stage (the ladder is branch arithmetic; the honest claim is correctness-under-adversary, which is what §4 carries). What exists is

measurement-as-evidence, regenerated by CI. Table rows quote or name the artifact that carries them; nothing here is hand-narrated from memory.

Check	Result	Artifact
Test suite	313 tests (0.5.0, "with the SPDX additions"; grew 260 → 306 → 313 across 0.4.x–0.5.0)	<code>CHANGELOG.md</code> §0.5.0 "tests" (line ~92); <code>README.md</code> receipts table; CI run per push
Python versions	3.12–3.14 matrix, both invocation styles	<code>README.md</code> receipts table; <code>.github/workflows/ci.yml</code>
Bounded ladder verification	28,080 cases, full finite space at length ≤ 3 , <code>all_invariants_pass: true</code> , 10 invariant checks I1–I10 PASS, deterministic reproduce command	<code>docs/receipts-ladder-verification.json</code>
Tamper soak	1,500 mutated ledgers × 5 seeds → 100% detected, 0 silent passes (nightly; push runs a 150-ledger slice — stated as such)	<code>README.md</code> receipts table; <code>CHANGELOG.md</code> §"Nightly soak workflow" line ~350
Canary protocol	10 catches / 3 honest misses / 0 false positives across 12 defect classes — the misses published	<code>README.md</code> receipts table
Spec parity (two implementations)	Reference verifier $\equiv$ spec-only verifier on 8 failure modes ; the spec-only verifier (<code>examples/spec_verifier.py</code>) imports nothing from the package	<code>README.md</code> receipts table; vectors README asserts build-time agreement of both ladders "on every case"
Cross-implementation ladder parity	Reference ladder and spec-only ladder agree on every contract-vector case, asserted at build time	<code>CHANGELOG.md</code> §0.4.x lines 325/341; <code>docs/standard-test-vectors/README.md</code> ; <code>tests/test_contract_vectors.py</code>
CI-enforced end-to-end parity	New harness run as a CI step: reference CLI $\equiv$ <code>spec_verifier</code> $\equiv$ pinned expectations on vectors and the dogfood trio	<code>scripts/crosscheck_verifier.py</code> ; <code>.github/workflows/ci.yml</code> step "Verifier parity..." (<code>CHANGELOG.md</code> Unreleased)
Real-LLM jury run	Live model produced doctrine on both dogfood claims (real rationales, honest low confidence where digests can't decide); certificate verifies offline (<code>valid/chain_valid/signature_valid/verdicts_match</code> all true, risk low, score 1.0); first GATE-mode attempt showed fail-closed catching real-vs-stub divergence (<code>divergence-split</code> , <code>inconclusive-unresolved</code> , GATE blocked)	<code>CHANGELOG.md</code> §0.3.2; workflow <code>real-jury-receipt.yml</code> (manual dispatch, key via repo secret, never committed); GitHub Actions run 34787467296
Dogfood end-to-end	Repository audits itself: signed self-audit certificate regenerates its own badge on every push (worst-verdict-wins), offline-replayable	<code>README.md</code> receipts intro; <code>dogfood_ledger.jsonl</code> + <code>dogfood_cert.json</code> + release assets
Anchor dogfood	v0.5.0 run anchored to Rekor (sidecar shape in §3.5), verify path exercised	<code>docs/receipts-anchor-v050-dogfood.json</code>
Marketplace e2e	Action proven through the public <code>uses: ./</code> path on every tag; v0.5.0: success; anchor inputs live	<code>docs/notes/2026-09-15-landscape-refresh.md</code> §"marketplace bar"
SPDX interop	<code>export --format spdx</code> → SPDX 3.0.1 AI-profile JSON-LD validated against the pinned official schema	<code>CHANGELOG.md</code> §0.5.0 "export"

Check	Result	Artifact
	(schema under <code>docs/data/</code> , sha256-locked in tests); deterministic; refusal to map risk into fields SPDX doesn't define is <i>test-locked</i>	
Watcher ecosystem self-dogfood	11 shipped example watchers (secret-scan, license-scan, docker-hygiene, doc-sync, SBOM, ally, import-weight, security, compliance, cost, aiact-transparency) held to the same conformance kit a third party faces; the launch post said "ten ... at the moment" (2026-09-13) — aiact-transparency landed with the regulatory wedge after it	<code>examples/manifests/</code> (11 JSON) + <code>watchers/</code> (11 py), counted on disk 2026-09-15
Supply-chain posture	OpenSSF Scorecard in CI; badge live	<code>README.md</code> ; <code>.github/workflows/</code>

A deliberate silence: **no wall-clock or scale benchmarks**. The workload being audited is someone else's CI; the protocol's cost profile is a single extra verify pass, and any number I would print here would be a demo-laptop number. When production adopters exist (§7), they will publish better ones.

6. Implementation notes worth stating

Three engineering decisions generalize beyond this project, each already cited by path:

1. **Encode bounded data as one string, not a list literal.** 28k-tuple literals die in elaborators (any language's); a 337KB digit string plus a trivial decoder survives and stays kernel-checkable (§4.2; `TruthTable.lean` header comment records the lesson).
2. **Split case-with-quantifiers before decide.** The oracle's 15-line decoder + per-row `Bool` check makes the 28,080-way conjunction a single `native_decide` target (§4.2, Ladder header "case-split anything with quantifiers").
3. **Pin the oracle to shipped artifacts, not to a test harness.** The Lean table is regenerated from the same code path that computes the digest in the release receipt; CI green $\Rightarrow$ the *published* behavior was re-checked, not the test-fixture behavior (§4, "fidelity rests on two receipts").

Core dependencies: `cryptography>=42`, `httpx>=0.27` — nothing else in the runtime; test extras add only `pyyaml/jsonschema`. Small TCB is a feature with a citation path: `pyproject.toml`.

7. Limitations and exit criteria

Honest inventory, in descending order of significance:

1. **The standard's own exit criteria are external by nature** (design §8, Phase 3): ① a third party implements an independent verifier from the spec — issue #1, open; the closure kit (`docs/verifier-onboarding.md`: Tier-0 trusted-artifact table, nine byte-level obligations with the traps that bit historically, one-command parity harness, proofs-as-behavioral-spec) is in-tree precisely to make this someone else's shortest possible errand; ② first external production deployment — none yet; ③ ≥ 10 active third-party watcher manifests — self-dogfooded only (all 11 example watchers are mine). Until ①–③ fill, "standard" means "normative draft with published errata," and this paper says so.
2. **Computational boundary:** ECDSA (P-256/Rekor) and SHA-256 soundness are assumed, never discharged (§4.4, §4.5); a proof-toolchain crypto lane (`cryptoProofs/EasyCrypt` territory) or permanent

TCB disclosure is the fork in the road, and the repo chose disclosure plus the architecture theorems that survive it (A4/A0).

3. **Model-of, not verification-of:** the Lean lanes verify abstract models whose fidelity to Python is pinned by the oracle + source digests *inside the enumerated domain*; outside it, fidelity is inspection-grade (Ladder.lean header, verbatim).
 4. **Statistical jury risk:** family-diversity defenses fail the way they are designed against — correlated training lineage; the calibration ledger discounts, W1a supremacy constrains, but no cryptographic statement is possible about independently thinking models (design §7.4 risk 4, honest note quoted).
 5. **Single-maintainer bus factor:** sole author. GOVERNANCE.md states the target pattern (SARIF/ Kubernetes: open standard, operational layer above it, foundation transfer if standard status settles) but there is no second key-holder today — stated because an audit protocol's trust story cannot ignore who can silently *stop*.
 6. **Publication hygiene as policy:** numbers carry dates because they will move — the project's convention is amend-forward, never rewrite ("Receipts amended forward rather...", CHANGELOG.md §0.5.0 line ~92); this paper inherits it (README/CHANGELOG test-count drift was corrected in-repo by e01b54e while this draft was written — that correction is itself the pattern working).
 7. **The measurement admits its misses:** 3 canary defect classes went undetected and are published (10/3/0), because a catch-everything measurement is a rigged measurement (design §13; README).
-

8. Artifact availability

- **Source:** github.com/goun7/veridict, Apache-2.0, all history public; release v0.5.0 (tag v0.5.0).
 - **Distribution:** PyPI veridict-standard (pip install veridict-standard; the bare veridict name is an unrelated project — the README install line says so).
 - **GitHub Action:** "veridict audit", Marketplace-listed, exercised via uses: ./ on tags (action.yml; landscape note §marketplace).
 - **Standard & vectors:** docs/specs/2026-09-10-veridict-standard-v1.0.md; docs/standard-test-vectors/ (ledger.jsonl, certificate.json, expected_verify.json, ladder/watcher vectors, generator README).
 - **Proofs reproducibility** (each from its lane directory, toolchain pinned by lean-toolchain = leanprover/lean4:v4.34.0):
bash # lane 1+2 (ladder theorems + 28,080-row oracle): cd proofs/ladder && lake build Ladder TruthTable # lanes 3+4 (chaining + anchor structure), axiom audits: cd proofs/ledger && lake build && lake env lean axiom_audit.lean cd proofs/anchor && lake build && lake env lean axiom_audit.lean # oracle regeneration (digest-tied) + bounded receipt: python3 scripts/export_lean_truth_table.py && python3 scripts/verify_ladder.py
 - **CI workflows:** .github/workflows/proofs.yml ("Formal proofs" — four kernel-check lanes, three axiom audits tee'd to the run summary, one trust-placement disclosure step), ci.yml (test suite, dogfood self-audit, offline replay, Verifier-parity step), real-jury-receipt.yml (manual, run 34787467296), nightly soak.
 - **Onboarding kit:** docs/verifier-onboarding.md + scripts/crosscheck_verifier.py (--extra your_verifier.py to fold a candidate implementation into the same parity net; maintainer runs accepted candidates in CI).
-

References

Only artifacts actually cited above, in text order. arXiv identifiers re-grounded 2026-09-15 as recorded in the landscape note; secondary surveys not reproduced (cf. the AegisChain verification failure documented there).

1. **arXiv:2607.09682** — AuditWeave. (Ledger-primitive convergence, §2.)
 2. **arXiv:2608.22512** — HANSARD: agent-activity recordkeeping; "records produced by suspects don't acquit suspects." (§1.2, §2.)
 3. **arXiv:2606.23003** — VCT. (Ledger-primitive convergence, §2.)
 4. **arXiv:2604.24890**, **arXiv:2603.02378** — C2PA in-band provenance gap analyses. (§2.)
 5. **arXiv:2609.12002**, **arXiv:2609.12439**, **arXiv:2606.13221**, **arXiv:2607.28636** — judge calibration / jury heterogeneity cluster. (§1.2, §2.)
 6. **arXiv:2510.10185** — MedAgentAudit (98.42% self-consistency collapse). (§2.)
 7. **microsoft/agent-governance-toolkit** — GitHub, runtime agent governance (~6.3k★ at grounding date). (§2.)
 8. **SLSA / VSA** — slsa.dev, build provenance & verification attestations. (§2.)
 9. **EU AI Act** — GPAI obligations & Art. 50 transparency live 2026-08-02; Omnibus marking grace ends 2026-12-02 (dates per landscape note §"Regulatory clock"). (§1.1.)
 10. **Verdict standard v1.0.0 (draft, errata)** — docs/specs/2026-09-10-veridict-standard-v1.0.md.
 11. **Verdict design document** — docs/specs/2026-09-09-veridict-design.md (§1, §5.2, §7.4, §8).
 12. **Landscape & position note (verified 2026-09-15)** — docs/notes/2026-09-15-landscape-refresh.md.
 13. **Proof lanes** — proofs/ladder/Ladder.lean, proofs/ladder/TruthTable.lean, proofs/ledger/Chain.lean, proofs/anchor/Anchor.lean (headers carry the measured axiom blocks quoted in §4).
 14. **Bounded receipt** — docs/receipts-ladder-verification.json; **anchor dogfood** — docs/receipts-anchor-v050-dogfood.json.
 15. **Conformance vectors** — docs/standard-test-vectors/ (incl. expected_verify.json, ladder_vectors.json, README).
 16. **Spec-only verifier** — examples/spec_verifier.py; **parity harness** — scripts/crosscheck_verifier.py; **verifier onboarding kit** — docs/verifier-onboarding.md.
 17. **CI** — .github/workflows/{proofs.yml, ci.yml, real-jury-receipt.yml}; jury run 34787467296.
 18. **Release history** — CHANGELOG.md (§0.3.2 jury receipt; §0.5.0 proofs/export/tests lines cited in §4–§5; Unreleased anchor-lane entry).
 19. **Repository README** — README.md (receipts table quoted in §5).
-